

分布式系统经典论文学习版

MapReduce：大型集群上的 简化数据处理

完整中文翻译 + 学习导读

原文：Jeffrey Dean 与 Sanjay Ghemawat, Google Inc.

发表于 OSDI 2004

阅读目标：理解一个受限编程模型如何把并行化、容错、调度、数据本地性和尾延迟治理封装进运行时。

学习导读：先建立系统图景

这篇论文提出的 MapReduce 是一个编程模型及其运行时实现。它把用户真正关心的“如何从一条记录产出中间结果”和“如何汇总同一键的结果”，与分布式系统必须处理的调度、通信、故障、负载不均分离开来。论文中的实现面向 Google 当时由数百到数千台廉价服务器组成的集群。

一页梗概

问题	论文给出的回答
目的	让普通程序员能在大规模廉价机器集群上处理 TB 级数据，而不必手写并行、容错和通信代码。
抽象	用户只实现 $\text{map}(k1, v1) \rightarrow \text{list}(k2, v2)$ 与 $\text{reduce}(k2, \text{list}(v2)) \rightarrow \text{list}(v2)$ 。
主线	Map 产生中间键值对；系统按键分区、搬运、排序并分组；Reduce 汇总同键全部值。
可靠性	任务输出写入私有临时文件，以原子提交确定唯一有效结果；失败任务自动重试。
性能	尽可能把 Map 调度到数据副本所在机器；大量细粒度任务动态均衡；临近结束时投机执行慢任务。
影响	把大规模批处理工程化，直接影响后来的 Hadoop 生态与数据处理系统设计。

你应该带着什么目的读

不是背 API：把它当作一个分布式系统的“分层案例”。用户函数的语义很小，运行时的职责却很大。

重点观察边界：用户负责业务逻辑；Master 负责任务状态与调度；Worker 负责执行；GFS 提供输入与最终输出的可靠存储。

重点观察取舍：Map 的中间结果只写本地盘，减少网络写入，却使已完成 Map 的机器宕机后必须重跑。

重点观察尾延迟：总作业完成时间由最慢任务决定。备份任务（投机执行）用少量额外资源消除拖后腿者。

核心原理：从函数到分布式流水线

Map 的输出不是最终结果，而是可按键重组的中间数据。运行时先用 `partition(key)` 把数据划给 R 个 Reduce 分区，再让每个 Reduce 拉取属于自己的中间数据、按键排序、把同一键的所有值交给用户的 Reduce。这个“按键重分布 + 分组”过程通常称为 shuffle。

阶段	系统做什么	用户代码做什么
输入切分	把输入文件切成 M 个 split，并尽量安排在有本地副本的机器。	无。
Map	调度任务、缓存并落盘中间数据、按 Reduce 分区。	逐条输入记录发射中间 <code><key, value></code> 。
Shuffle / Sort	Reduce Worker 远程拉取数据，外部排序并按 key 分组。	无。
Reduce	为每个唯一键提供值迭代器，写入该分区最终文件。	合并该键的一组值并发射结果。
提交与恢复	原子提交、故障检测、重试、投机任务与状态展示。	保持函数确定性；副作用须幂等且原子。

架构与数据流

可以用以下顺序理解图 1：

编号	数据与控制流
1	用户程序启动许多进程，其中 1 个为 Master，其余为 Worker；输入被切成 M 个 split。
2	Master 将空闲 Map 或 Reduce 任务分派给空闲 Worker。
3	Map Worker 读取自己的 split，调用用户 Map，得到中间键值对。
4	中间对缓存后写入本机磁盘的 R 个分区，位置报告给 Master。
5	Reduce Worker 从所有 Map Worker 远程读取属于自己分区的数据，随后排序分组。
6	Reduce Worker 调用用户 Reduce，向一个最终输出文件追加结果。

实现逻辑与关键不变量

机制	为何这样设计	应记住的结论
原子提交	重复执行时必须只接受一个结果。Map 上报首次完成的 R 个临时文件；Reduce 将临时文件原子重命名为最终文件。	确定性的 Map/Reduce 在故障下等价于一次无故障顺序执行。
数据本地性	网络是稀缺资源；GFS 块有多副本。	把计算迁到数据旁，常比把数据搬到计算旁更重要。
细粒度任务	M、R 大于 Worker 数量，快机器能多做任务，失败恢复可分散。	细粒度提升动态负载均衡，却增加 Master 元数据和调度开销。
Combiner	许多 Map 会在同一键上产生大量可局部合并的值。	先局部聚合可显著减少网络 shuffle；前提是操作满足适用的结合/交换性质。
备份任务	最后少数慢任务形成长尾。	作业接近结束时复制剩余任务，谁先完成就采用谁的结果。

阅读后的自测

Q1：为什么完成的 Map 任务在机器失效后要重跑，而完成的 Reduce 不必？ 因为 Map 中间文件在该 Worker 的本地盘；Reduce 最终文件写在全局文件系统。

Q2：为什么 Reduce 还要排序？ 许多不同键会落入同一个 Reduce 分区；排序将相同键相邻，才可按键把完整值集合交给 Reduce。

Q3：为什么 MapReduce 很适合批处理却不适合低延迟在线事务？ 它通过文件、批量重分布和全局阶段完成来换吞吐与可恢复性，天然有批处理屏障和 I/O 延迟。

论文完整中文翻译

译者说明：以下按原文结构完整翻译正文、图表说明、致谢、参考文献与附录。为保持技术准确性，Map、Reduce、Master、Worker、shuffle、split、Combiner 等常用术语保留英文或采用中英对照。

题目与作者

MapReduce：大型集群上的简化数据处理

Jeffrey Dean, Sanjay Ghemawat

Google 公司

摘要

MapReduce 是一种用于处理和生成大型数据集的编程模型及其配套实现。用户指定一个 map 函数，它处理一个键/值对并生成一组中间键/值对；再指定一个 reduce 函数，它合并与同一中间键相关联的全部中间值。正如本文所示，许多现实世界任务都能够用这种模型表达。

采用这种函数式风格编写的程序，会自动并行化并在大型廉价机器集群上执行。运行时系统负责输入数据分片、将程序执行调度到一组机器、处理机器故障，以及管理必需的机器间通信等细节。因此，即使没有并行与分布式系统经验的程序员，也能方便地利用大型分布式系统的资源。

我们的 MapReduce 实现在大型廉价机器集群上运行，并且具有很高的可扩展性：一次典型 MapReduce 计算会在数千台机器上处理数 TB 数据。程序员发现该系统易于使用：已经实现了数百个 MapReduce 程序，Google 集群每天执行超过一千个 MapReduce 作业。

1 引言

过去五年里，作者以及 Google 的许多人实现了数百种专用计算，用来处理大量原始数据，例如抓取到的文档、Web 请求日志等，以计算各种派生数据，例如倒排索引、网页文档图结构的多种表示、每个主机抓取页面数量的摘要、某一天最常见的查询集合等。多数这类计算在概念上很直接。然而，输入数据通常很大，计算必须分布到数百或数千台机器上，才能在合理时间内结束。

如何并行化计算、如何分发数据、如何处理故障等问题，使原本简单的计算被处理这些问题的大量复杂代码淹没。

为应对这种复杂性，我们设计了一种新抽象：它让我们表达所需的简单计算，同时把并行化、容错、数据分布和负载均衡的繁杂细节隐藏在一个库中。该抽象受 Lisp 以及许多其他函数式语言中的 map 和 reduce 原语启发。我们发现，大多数计算都包含：对输入中的每条逻辑“记录”施加 map 操作，以计算一组中间键/值对；随后对共享同一键的全部值施加 reduce 操作，以恰当地合并派生数据。用户指定 map 与 reduce 操作的函数式模型，使我们能够轻松并行化大规模计算，并把重新执行作为主要容错机制。

这项工作的主要贡献是：一个简单而强大的接口，能够自动并行化和分发大规模计算；以及该接口的一种实现，它能在大型廉价 PC 集群上实现高性能。第 2 节描述基本编程模型并给出若干示例；第 3 节描述为集群计算环境定制的 MapReduce 接口实现；第 4 节给出若干实用扩展；第 5 节测量多种任务上的性能；第 6 节探讨 Google 内部的使用情况，包括用它重写生产索引系统的经验；第 7 节讨论相关和未来工作。

2 编程模型

一次计算接收一组输入键/值对，并产生一组输出键/值对。MapReduce 库的用户把计算表达为两个函数：Map 与 Reduce。

用户编写的 Map 接收一个输入对，并产生一组中间键/值对。MapReduce 库将关联到同一中间键 k 的全部中间值归在一起，并传给 Reduce 函数。

同样由用户编写的 Reduce 接收一个中间键 k 和该键的一组值。它合并这些值，形成可能更小的一组值。通常每次 Reduce 调用只产生 0 个或 1 个输出值。中间值通过迭代器提供给用户的 reduce 函数，因此即使值列表大到放不进内存也可以处理。

2.1 示例

考虑在大型文档集合中统计每个词出现次数的问题。用户会写出类似如下的伪代码：

```
map(String key, String value):  
  // key: 文档名; value: 文档内容  
  对 value 中的每个词 w:  
    EmitIntermediate(w, "1")
```

```

reduce(String key, Iterator values):
  // key: 一个词; values: 计数列表
  result = 0
  对 values 中的每个 v:
    result += ParseInt(v)
  Emit(AsString(result))

```

map 函数为每个词发射该词及其关联出现次数（这个简单例子中就是“1”）。reduce 函数把为某个特定词发射的全部计数相加。除此之外，用户还要编写代码，在 mapreduce 规格对象中填入输入、输出文件名以及可选的调优参数；随后把该规格对象传给 MapReduce 函数。用户代码与 MapReduce 库（以 C++ 实现）链接。附录 A 给出了这一示例的完整程序。

2.2 类型

尽管之前的伪代码用字符串输入输出表示，但从概念上说，用户提供的 map 和 reduce 函数具有如下类型：

```

map (k1, v1) -> list(k2, v2)
reduce (k2, list(v2)) -> list(v2)

```

也就是说，输入键和值来自不同于输出键和值的域；而中间键和值与输出键和值来自同一域。我们的 C++ 实现向用户定义的函数传入和传出字符串，由用户代码负责在字符串与适当类型间转换。

2.3 更多示例

分布式 Grep：map 函数在一行匹配给定模式时发射该行。reduce 函数是恒等函数，只把给定中间数据复制到输出。

URL 访问频率计数：map 处理网页请求日志并输出 <URL, 1>；reduce 对同一 URL 的所有值求和并输出 <URL, 总计数>。

反向 Web 链接图：对页面 source 中发现的每个指向 target URL 的链接，map 输出 <target, source>。reduce 拼接关联给定 target URL 的全部 source URL 列表，输出 <target, list(source)>。

每台主机的词项向量：词项向量以 <word, frequency> 对的列表概括一个文档或一组文档中最重要词。map 为每个输入文档发射 <hostname, term vector>（hostname 从文档 URL 提

取)；reduce 接收给定主机的所有逐文档词项向量，将其相加、丢弃低频项，并发射最终 `<hostname, term vector>`。

倒排索引：map 解析每个文档并发射一系列 `<word, document ID>` 对。reduce 接收给定词的全部对，对相应文档 ID 排序并发射 `<word, list(document ID)>`。所有输出对构成简单倒排索引；很容易扩展为记录词位置。

分布式排序：map 从每条记录提取键并发射 `<key, record>`；reduce 原样发射全部对。该计算依赖第 4.1 节的分区能力与第 4.2 节的顺序性质。

3 实现

MapReduce 接口可以有许多不同实现，正确选择取决于环境：一种实现可能适合小型共享内存机器，另一种适合大型 NUMA 多处理器，还有一种适合更大的网络化机器集合。本节描述面向 Google 广泛使用的计算环境的实现：通过交换式以太网连接的大型廉价 PC 集群。

环境特征	原文描述
机器	通常为运行 Linux 的双处理器 x86，单机 2-4 GB 内存。
网络	使用商用网络硬件；机器级通常 100 Mb/s 或 1 Gb/s，但总体二分带宽平均明显更低。
故障	集群有数百或数千台机器，因此机器故障很常见。
存储	每机直连廉价 IDE 磁盘；内部开发的分布式文件系统管理数据，用复制在不可靠硬件上提供可用性和可靠性。
作业提交	用户向调度系统提交作业；每个作业由一组任务组成，调度器将其映射到集群中的可用机器。

3.1 执行概览

Map 调用通过将输入数据自动划分为 M 个 split 而分布到多台机器，不同机器可并行处理不同 split。Reduce 调用则通过分区函数把中间键空间划为 R 份而分布，例如 $\text{hash}(\text{key}) \bmod R$ 。分区数 R 与分区函数由用户指定。图 1 给出了整体流程；当用户程序调用 MapReduce 函数时，依次发生以下动作：

用户程序中的 MapReduce 库先把输入文件切为 M 块，通常每块 16 MB 到 64 MB（用户可用可选参数控制），再在集群机器上启动多个程序副本。

其中一个程序副本特殊，它是 Master；其余为 Worker，由 Master 分配工作。共有 M 个 map 任务与 R 个 reduce 任务等待分配。Master 选择空闲 Worker，为每个 Worker 分派一个 map 或 reduce 任务。

分到 map 任务的 Worker 读取对应输入 split 的内容，从输入中解析键/值对，逐对调用用户定义的 Map 函数。Map 产生的中间键/值对先在内存中缓冲。

缓冲对会周期性写到本地磁盘，并通过分区函数分成 R 个区域。这些本地磁盘文件的位置回传给 Master；Master 负责把位置转发给 reduce Worker。

Reduce Worker 从 Master 得知位置后，以远程过程调用从 map Worker 的本地磁盘读取缓冲数据。读完全部中间数据后，它按中间键排序，让同一键的全部出现项聚在一起。通常多个不同键会映射到同一 reduce 任务，因此需要排序；若中间数据大到内存放不下，则使用外部排序。

Reduce Worker 遍历已排序的中间数据；对遇到的每个唯一中间键，把该键和相应的一组中间值交给用户的 Reduce 函数。Reduce 输出会追加到这个 reduce 分区的最终输出文件。

全部 map 与 reduce 任务完成后，Master 唤醒用户程序；此时用户程序中的 MapReduce 调用返回。

成功结束后，MapReduce 执行的结果位于 R 个输出文件中（每个 reduce 任务一个文件，文件名由用户指定）。通常用户不需要把 R 个输出合并为一个文件：这些文件常直接作为另一次 MapReduce 调用的输入，或供能处理多文件分区输入的其他分布式应用使用。

3.2 Master 数据结构

Master 保持若干数据结构。对每个 map 与 reduce 任务，它保存状态（空闲、进行中或已完成），以及 Worker 机器身份（非空闲任务）。Master 还是把中间文件区域位置从 map 任务传播到 reduce 任务的通道。因此，对每个已完成 map 任务，Master 都保存该任务产生的 R 个中间文件区域的位置和大小。map 任务完成时收到这些位置和大小更新；信息会渐进地推送给正在进行的 reduce 任务的 Worker。

3.3 容错

MapReduce 库面向在数百或数千台机器上处理极大量数据的场景，因此必须优雅地容忍机器故障。

Worker 故障：Master 周期性 ping 每个 Worker。如果在一定时间内未收到响应，Master 将该 Worker 标记为失效。该 Worker 已完成的 map 任务恢复为初始空闲状态，因而可在其他 Worker 上调度；失效 Worker 上进行中的 map 或 reduce 任务也恢复为空闲并可重新调度。已完成的 map 必须重执行，因为输出在失效机器本地盘、不可访问；已完成 reduce 不需重执行，因为输出在全局文件系统。若 map 先在 Worker A、后因 A 失效在 Worker B 执行，正在执行 reduce 的 Worker 会收到重执行通知；尚未从 A 读取数据的 reduce 任务会改从 B 读取。该机制可承受大规模故障：论文给出的案例中，网络维护使每次 80 台机器数分钟不可达，Master 仅重新执行这些机器完成的工作，仍持续前进并最终完成作业。

Master 故障：可让 Master 对上述数据结构周期性写检查点；若 Master 任务死亡，可从最近检查点启动新副本。但由于仅有一个 Master，其失败不太可能，论文当时的实现选择在 Master 失败时中止 MapReduce 计算；客户端可检测并按需重试整个作业。

故障下的语义：若用户提供的 map 与 reduce 算子是其输入值的确定性函数，分布式实现产生的输出与整个程序一次无故障顺序执行的输出相同。该性质依赖任务输出的原子提交：每个进行中任务写自己的临时私有文件；reduce 任务写一个，map 任务写 R 个。map 完成时把 R 个临时文件名发给 Master；Master 若已收到该 map 的完成消息就忽略，否则记录文件名。reduce 完成时，Worker 把临时输出文件原子重命名为最终文件；同一 reduce 多机执行时可能有多个 rename，但底层文件系统的原子重命名保证最终状态只含其中一次执行产生的数据。

绝大多数 map 与 reduce 算子是确定性的，这种“等价于顺序执行”的语义使程序员很容易推理。算子非确定时，系统提供较弱但仍合理的语义：某个特定 reduce 任务 R1 的输出，等价于非确定程序某次顺序执行中 R1 的输出；但另一个 reduce 任务 R2 的输出，可能对应另一种顺序执行。这是因为已提交的 R1 执行可能读取 map 任务 M 的某次执行输出，而已提交的 R2 执行可能读取 M 的另一次执行输出。

3.4 数据本地性

在该环境中，网络带宽相对稀缺。系统利用 GFS 管理的输入数据已存储在集群机器本地磁盘这一事实来节约网络带宽。GFS 将每个文件分成 64 MB 块，并将每块的多个副本（通常 3 份）存于不同机器。MapReduce Master 会考虑输入文件的位置信息，优先把 map 任务调度到含相应输入数据副本的机器；若做不到，则尝试调度到副本附近（例如与数据所在机器连接到同一网络交换机的 Worker）。在使用集群中相当大比例 Worker 的大型 MapReduce 作业中，大多数输入数据从本地读取，不消耗网络带宽。

3.5 任务粒度

如前所述，Map 阶段划为 M 块，Reduce 阶段划为 R 块。理想情况下，M 与 R 应明显大于 Worker 机器数。让每个 Worker 执行很多不同任务可改善动态负载均衡，也能加快 Worker 失效后的恢复：它完成的众多 map 任务可分散到所有其他 Worker。M 与 R 的大小有限，因为 Master 必须做 $O(M + R)$ 次调度决策，并保留 $O(M * R)$ 的内存状态；不过内存常数较小， $O(M * R)$ 那部分状态对每个 map/reduce 任务对大约仅 1 字节。

此外，R 常受用户约束，因为每个 reduce 的输出会成为一个单独文件。实践中，作者让每个 map 任务处理约 16 MB 到 64 MB 输入，以最大化前述本地性优化；R 取为预计使用 Worker 数的小倍数。论文常见配置是 $M = 200,000$ 、 $R = 5,000$ ，使用 2,000 台 Worker。

3.6 备份任务

拖慢 MapReduce 总耗时的一个常见原因是“落后者”（straggler）：它异常久地完成计算中最后几个 map 或 reduce 任务之一。落后者可能有许多原因：坏磁盘频繁发生可纠正错误，使读取速度从 30 MB/s 降到 1 MB/s；集群调度器在机器上安排其他任务，争用 CPU、内存、本地盘或网络带宽；作者还遇到过机器初始化代码 bug 禁用处理器缓存，使受影响机器计算慢一百多倍。

为缓解落后者问题，作业接近完成时，Master 为剩余进行中任务安排备份执行。原始或备份执行任一完成，任务即标记完成。该机制经过调优，通常只让作业使用的计算资源增加几个百分点，却显著缩短大作业完成时间。例如第 5.3 节排序程序若禁用备份任务，完成时间长 44%。

4 扩展

仅通过编写 Map 与 Reduce 函数提供的基本功能已满足大多数需求，但作者发现以下扩展也很有用。

4.1 分区函数

MapReduce 用户指定所需 reduce 任务/输出文件数 R 。数据依据中间键上的分区函数分配给这些任务。系统提供使用哈希的默认分区函数，例如 $\text{hash}(\text{key}) \bmod R$ ，通常能得到均衡分区。但有时按键的其他函数分区更有用。例如输出键是 URL 时，希望同一主机的所有条目进入同一输出文件。用户可提供特殊分区函数；使用 $\text{hash}(\text{Hostname}(\text{urlkey})) \bmod R$ ，就使来自同一主机的所有 URL 进入同一输出文件。

4.2 顺序保证

系统保证：在给定分区内，中间键/值对按键的递增顺序处理。这一保证使得每个分区都能生成已排序输出文件；当输出文件格式需要支持按键的高效随机访问查找，或者输出使用者觉得数据有序更方便时，这很有用。

4.3 Combiner 函数

有时每个 map 任务产生的中间键存在大量重复，而且用户指定的 Reduce 函数满足交换律和结合律。词频统计是很好的例子：由于词频通常服从 Zipf 分布，每个 map 任务会产生数百或数千条 $\langle \text{the}, 1 \rangle$ 记录。若直接发送，全部计数经网络传到一个 reduce 任务再相加为一个数。系统允许用户指定可选 Combiner，在数据经过网络前做部分合并。

Combiner 在执行 map 任务的每台机器上运行。通常 Combiner 和 Reduce 使用同一段代码；差别只在 MapReduce 库如何处理函数输出：Reduce 输出写到最终输出文件，Combiner 输出写到将发送给 reduce 任务的中间文件。部分合并可显著加快某些类别的 MapReduce 操作；附录 A 给出了使用 Combiner 的例子。

4.4 输入与输出类型

MapReduce 库支持读取多种输入格式。例如，“text”模式把每一行视为键/值对：键是文件偏移量，值是行内容。另一种常见支持格式存储按键排序的一系列键/值对。每种输入类型实现都知道如何将自身切成有意义的范围，作为独立 map 任务处理（例如 text 模式确保范围只在行边界切分）。用户可通过实现一个简单 reader 接口来支持新输入类型，但多数用户只使用少数预定义类型。Reader 不必从文件读取数据，例如可定义从数据库或内存映射数据结构读取记录的 reader。类似地，系统支持以不同格式产生数据的一组输出类型，用户代码也很容易添加新输出类型。

4.5 副作用

有时用户发现，从 map 和/或 reduce 算子产生辅助文件作为额外输出很方便。系统要求应用编写者使这类副作用具备原子性与幂等性；通常应用先写临时文件，完整生成后再原子重命名。系统不支持单个任务产生的多个输出文件的原子两阶段提交。因此，若多个输出文件之间有跨文件一致性要求，产生这些文件的任务应当是确定性的。实践中这一限制从未成为问题。

4.6 跳过坏记录

有时用户代码的 bug 会使 Map 或 Reduce 函数在某些记录上确定性崩溃，从而阻止作业完成。通常应修复 bug，但有时不可行，例如 bug 位于没有源码的第三方库；在大数据集统计分析中，忽略少数记录有时也可接受。系统提供可选执行模式：检测导致确定性崩溃的记录并跳过它们，从而继续前进。

每个 Worker 进程都安装捕获段错误和总线错误的信号处理器。调用用户 Map 或 Reduce 前，库将参数的序列号存到全局变量。若用户代码产生信号，处理器向 Master 发送包含该序列号的“临终”UDP 包。当 Master 对某记录已见到多次失败，就会在下次重新执行相应 Map 或 Reduce 任务时指示跳过该记录。

4.7 本地执行

调试 Map 或 Reduce 问题可能很棘手，因为实际计算发生在分布式系统中，常涉及数千台机器，且 Master 动态决定任务分配。为便于调试、性能剖析和小规模测试，作者开发了另一种

MapReduce 库实现：在本地机器上顺序执行一次 MapReduce 操作的全部工作。系统允许用户把计算限制到指定 map 任务；用户用特殊标志调用程序，便可使用任何调试或测试工具，例如 gdb。

4.8 状态信息

Master 运行内部 HTTP 服务器，导出供人查看的一组状态页。页面展示计算进度：已完成与进行中的任务数、输入字节数、中间数据字节数、输出字节数、处理速率等；也包含各任务产生的标准错误和标准输出文件链接。用户可据此预测计算还需多久、是否应增加资源，也可判断计算何时明显慢于预期。顶层状态页还展示失效 Worker，以及它们失效时正在处理的 map 与 reduce 任务；这有助于诊断用户代码 bug。

4.9 计数器

MapReduce 库提供计数器能力来统计各种事件，例如用户代码可能希望统计处理词总数或被索引的德语文档数。要使用它，用户代码创建命名计数器对象，并在 Map 和/或 Reduce 函数中适当递增。例如：

```
Counter* uppercase;  
uppercase = GetCounter("uppercase");  
map(String name, String contents):  
  对 contents 中的每个词 w:  
    若 IsCapitalized(w): uppercase->Increment();  
    EmitIntermediate(w, "1");
```

各 Worker 的计数器值会周期性传播给 Master（捎带在 ping 响应中）。Master 汇总成功 map 与 reduce 任务的计数器值，在作业结束时返回给用户代码；当前值也显示在 Master 状态页，便于观察实时计算。汇总时 Master 消除同一 map 或 reduce 任务重复执行的影响，避免重复计数（重复执行可能来自备份任务或故障后的重执行）。库还自动维护一些计数器，例如处理的输入键/值对数和产生的输出键/值对数。用户发现计数器很适合做 MapReduce 行为的合理性检查，例如确认输出对数恰等于处理的输入对数，或确认已处理的德语文档比例处于全部文档的可容忍范围。

5 性能

本节测量 MapReduce 在大型机器集群上运行两种计算时的性能：一种在约 1 TB 数据中搜索特定模式，另一种对约 1 TB 数据排序。这两种程序代表用户编写的现实程序中的很大子集：一类将数据从一种表示重排为另一种表示，另一类从大型数据集中提取少量感兴趣的数据。

5.1 集群配置

全部程序运行在约 1,800 台机器的集群上。每台机器有两个启用超线程的 2 GHz Intel Xeon 处理器、4 GB 内存、两块 160 GB IDE 磁盘和千兆以太网链路。机器构成两层树形交换网络，根部可用总带宽约 100-200 Gb/s；机器都在同一托管设施中，任意两台的往返时间小于 1 ms。4 GB 内存中约 1-1.5 GB 被集群运行的其他任务保留。程序在周末下午执行，当时 CPU、磁盘与网络大多空闲。

5.2 Grep

Grep 程序扫描 10^{10} 条、每条 100 字节的记录，搜索相对少见的三字符模式（该模式出现在 92,337 条记录中）。输入切成约 64 MB 的块（ $M = 15,000$ ），全部输出放到一个文件（ $R = 1$ ）。图 2 显示随时间变化的进度，纵轴为输入扫描速率。随着更多机器被分给此次计算，速率逐渐上升；当分到 1,764 个 Worker 时峰值超过 30 GB/s。map 任务完成后速率下降，在约 80 秒时归零。整个计算从开始到结束约 150 秒，其中约一分钟为启动开销：把程序传播到所有 Worker、与 GFS 交互打开 1,000 个输入文件、获取本地性优化所需信息均会造成延迟。

5.3 排序

排序程序对 10^{10} 条、每条 100 字节的记录排序（约 1 TB 数据），以 TeraSort 基准为原型。排序程序的用户代码少于 50 行：一个 3 行 Map 函数从文本行提取 10 字节排序键，并把该键和原始文本行作为中间键/值对发射；Reduce 使用内置 Identity 函数，原样把中间键/值对作为输出键/值对。最终排序结果写到一组双副本 GFS 文件，即程序输出写入 2 TB。

输入同样切成 64 MB 块（ $M = 15,000$ ），排序输出划为 4,000 个文件（ $R = 4,000$ ）。分区函数使用键的初始字节把记录分到 R 块之一。此基准的分区函数内建了对键分布的知识；一般排序程序可先做一次 MapReduce 预处理，收集键样本并用样本分布计算最终排序步骤的切分点。

图 3(a) 显示正常执行。左上图为输入读取速率，峰值约 13 GB/s，且在 200 秒前所有 map 已完成。输入速率低于 grep，因为排序 map 任务约一半时间与 I/O 带宽用于把中间输出写到本地盘；grep 的中间输出可忽略。左中图为数据从 map 到 reduce 经网络发送的速率。第一个 map 完成即开始 shuffle；图中第一个峰对应约 1,700 个 reduce 任务的第一批（整个作业约被分到 1,700 台机器，且每机同时最多执行一个 reduce）。约 300 秒时第一批中一些 reduce 完成，系统开始为剩余 reduce shuffle；约 600 秒时全部 shuffle 完成。

左下图为 reduce 向最终输出写入已排序数据的速率。第一段 shuffle 结束到开始写入之间有延迟，因为机器忙于排序中间数据。随后写入持续一段时间，速率约 2-4 GB/s；约 850 秒时全部写入完成。含启动开销，整个计算耗时 891 秒，接近当时 TeraSort 基准报告的最佳结果 1,057 秒。输入速率高于 shuffle 与输出速率，是因为本地性优化使大多数数据从本地盘读取，绕过了带宽较受限的网络。shuffle 速率高于输出速率，是因为输出阶段写入两份已排序数据以满足可靠性与可用性；若底层文件系统使用纠删码而非复制，写数据的网络带宽需求会更低。

5.4 备份任务的影响

图 3(b) 展示禁用备份任务的排序执行。流程与图 3(a) 类似，但尾部很长，几乎没有写活动。960 秒后，除 5 个 reduce 任务外其余都已完成；但最后几个落后者又过 300 秒才结束。整个计算用时 1,283 秒，经过时间增加 44%。

5.5 机器故障

图 3(c) 展示一次排序执行：计算开始数分钟后，作者有意杀死 1,746 个 Worker 进程中的 200 个。底层集群调度器立即在这些机器上重启新的 Worker 进程（杀死的是进程，机器仍正常）。Worker 死亡表现为负的输出速率，因为此前完成的一些 map 工作消失（相应 map Worker 被杀）并需重做。重执行很快发生。包括启动开销，整个计算 933 秒结束，仅比正常执行增加 5%。

6 经验

作者在 2003 年 2 月写出第一版 MapReduce 库，并在 2003 年 8 月做了重大增强，包括本地性优化、跨 Worker 的任务执行动态负载均衡等。此后，库在作者所处理问题中的适用广度令人惊

喜。它被用于：大规模机器学习问题；Google News 与 Froogle 产品的聚类问题；提取用于生成热门查询报告的数据（例如 Google Zeitgeist）；提取用于新实验与产品的网页属性（例如从大型网页语料提取地理位置以支持本地化搜索）；以及大规模图计算。

图 4 显示，主源码管理系统中单独 MapReduce 程序实例的数量显著增长：从 2003 年初的 0 增至 2004 年 9 月末近 900 个。MapReduce 成功，是因为它让人能写出简单程序，并在半小时内高效地在一千台机器上运行，极大加速开发和原型周期；它还让没有分布式/并行系统经验的程序员容易利用大量资源。每个作业结束时，库会记录所用计算资源统计。表 1 给出 Google 在 2004 年 8 月运行的一部分 MapReduce 作业统计。

指标	2004 年 8 月统计
作业数	29,423
平均作业完成时间	634 秒
使用的机器天数	79,186 天
读取输入数据	3,288 TB
产生中间数据	758 TB
写出输出数据	193 TB
每作业平均 Worker 数	157
每作业平均 Worker 死亡数	1.2
每作业平均 map 任务	3,351
每作业平均 reduce 任务	55
不同 map 实现	395
不同 reduce 实现	269
不同 map/reduce 组合	426

6.1 大规模索引

MapReduce 最重要的应用之一，是完全重写为 Google Web 搜索服务产生数据结构的生产索引系统。索引系统的输入是抓取系统检索到的大量文档，存为一组 GFS 文件；这些文档的原始内容超过 20 TB。索引过程由 5 到 10 次 MapReduce 操作组成。相较旧版索引系统中的临时分布式处理阶段，使用 MapReduce 有三项益处：

代码更简单：处理容错、分发和并行化的代码隐藏在 MapReduce 库中，索引代码更小、更易理解。例如某个计算阶段从约 3,800 行 C++ 降至用 MapReduce 表达后的约 700 行。

演进更快：库性能足够好，因此可让概念上无关的计算保持分离，而非为了避免额外扫描数据而混在一起。这让索引过程很容易修改：旧系统中需数月的一项修改，在新系统只需数天。

运维更容易：机器故障、慢机器和网络抖动导致的大多数问题都由库自动处理，无须操作员干预；向索引集群增加机器也很容易提升性能。

7 相关工作

许多系统都提供受限编程模型，并利用限制自动并行化计算。例如，一个结合函数可通过并行前缀计算，在 N 个处理器上用 $\log N$ 时间计算 N 元素数组的全部前缀。MapReduce 可被视为基于大型真实计算经验，对这类模型的简化与提炼；更重要的是，它提供了可扩展到数千处理器的容错实现。相比之下，多数并行处理系统只在较小规模实现，并把机器故障处理细节留给程序员。

大同步并行（BSP）和一些 MPI 原语提供更高层抽象，便于编写并程序。它们与 MapReduce 的关键区别是：MapReduce 利用受限编程模型，自动并行化用户程序并透明提供容错。它的数据本地性优化受到 active disks 等技术启发：把计算推到接近本地磁盘的处理单元，减少跨 I/O 子系统或网络的数据发送。该实现运行在每台直接连接少量磁盘的商用处理器上，而非直接运行在磁盘控制器处理器上，但总体思路类似。

备份任务机制类似 Charlotte 系统中的 eager scheduling。简单 eager scheduling 的缺点是，若特定任务反复失败，整个计算无法结束；MapReduce 用跳过坏记录机制修复了某些此类情形。实现还依赖内部集群管理系统，以在共享的大型机器集合上分发、运行用户任务；尽管不是本文焦点，该系统精神上类似 Condor。MapReduce 的排序能力与 NOW-Sort 类似：源机器（map Worker）对待排序数据分区并发送到 R 个 reduce Worker 之一；每个 reduce Worker 在本地排序（可能时在内存中）。当然，NOW-Sort 不具有用户可定义的 Map 与 Reduce 函数，后者使本库适用范围更广。

River 提供了进程通过分布式队列发送数据来通信的编程模型。与 MapReduce 一样，River 试图在异构硬件或系统扰动造成的不均匀性下保持良好的平均性能；它通过仔细调度磁盘和网络传

输以平衡完成时间。MapReduce 路径不同：受限模型使框架可把问题切成大量细粒度任务，动态调度给可用 Worker，使快机器处理更多任务；模型还允许在作业尾部调度任务的冗余执行，显著降低慢或卡住 Worker 带来的完成时间。BAD-FS 的模型很不同，面向广域网任务执行，但二者都用冗余执行从故障造成的数据丢失恢复，也都用感知本地性的调度减少拥塞网络链路上的数据传输。TACC 则旨在简化高可用网络服务构建，与 MapReduce 一样依赖重新执行实现容错。

8 结论

MapReduce 编程模型已在 Google 成功用于许多不同目的。作者将其成功归于三点。第一，模型易用：即使没有并行和分布式系统经验，程序员也可使用，因为它隐藏了并行化、容错、本地性优化和负载均衡细节。第二，广泛问题都能轻松表达为 MapReduce 计算，例如生成 Google 生产 Web 搜索服务的数据、排序、数据挖掘、机器学习以及其他许多系统。第三，作者实现了可扩展到由数千台机器组成的大型集群的 MapReduce；实现有效利用机器资源，适合 Google 遇到的许多大计算问题。

作者从这项工作学到：第一，限制编程模型使计算易于并行化、分发并实现容错。第二，网络带宽是稀缺资源，因此多个优化都在减少网络传输：本地性优化允许从本地磁盘读取数据，而把中间数据单份写入本地盘可节约网络带宽。第三，冗余执行既可减轻慢机器影响，也可处理机器故障和数据丢失。

致谢

Josh Levenberg 根据其使用 MapReduce 的经验及他人的增强建议，在修订和扩展用户级 MapReduce API、增加若干新功能方面发挥了关键作用。MapReduce 从 Google File System (GFS) 读取输入并向其写出输出；作者感谢 Mohit Aron、Howard Gobioff、Markus Gutschke、David Kramer、Shun-Tak Leung 和 Josh Redstone 在开发 GFS 中的工作；感谢 Percy Liang 和 Olcan Sercinoglu 开发 MapReduce 所用集群管理系统；感谢 Mike Burrows、Wilson Hsieh、Josh Levenberg、Sharon Perl、Rob Pike 和 Debby Wallach 对早

期草稿提出有益意见；感谢匿名 OSDI 审稿人和 shepherd Eric Brewer 提出许多改进建议；最后感谢 Google 工程组织内全部 MapReduce 用户提供有益反馈、建议和 bug 报告。

参考文献（译名）

- [1] Arpaci-Dusseau 等。《工作站网络上的高性能排序》。ACM SIGMOD, 1997。
- [2] Arpaci-Dusseau 等。《使用 River 的集群 I/O：让快速路径成为常态》。IOPADS, 1999。
- [3] Baratloo 等。《Charlotte：Web 上的元计算》。并行与分布式计算系统国际会议, 1996。
- [4] Barroso、Dean、Holzle。《面向整个星球的 Web 搜索：Google 集群架构》。IEEE Micro, 2003。
- [5] Bent 等。《面向批处理的分布式文件系统显式控制》。NSDI, 2004。
- [6] Blelloch。《作为基本并行操作的扫描》。IEEE Transactions on Computers, 1989。
- [7] Fox 等。《基于集群的可扩展网络服务》。SOSP, 1997。
- [8] Ghemawat、Gobioff、Leung。《Google 文件系统》。SOSP, 2003。
- [9] Gorlatch。《扫描及其他列表同态的系统化高效并行化》。Euro-Par, 1996。
- [10] Gray。《排序基准主页》。Microsoft Research。
- [11] Gropp、Lusk、Skjellum。《使用 MPI：基于消息传递接口的可移植并行编程》。MIT Press, 1999。
- [12] Huston 等。《Diamond：交互式搜索中用于早期丢弃的存储架构》。FAST, 2004。
- [13] Ladner、Fischer。《并行前缀计算》。Journal of the ACM, 1980。
- [14] Rabin。《用于安全、负载均衡与容错的信息高效分散》。Journal of the ACM, 1989。
- [15] Riedel 等。《用于大规模数据处理的活动磁盘》。IEEE Computer, 2001。
- [16] Thain、Tannenbaum、Livny。《实践中的分布式计算：Condor 经验》。Concurrency and Computation, 2004。
- [17] Valiant。《并行计算的桥接模型》。Communications of the ACM, 1990。
- [18] Wyllie。《Spsort：如何快速排序一个 TB》。IBM Almaden。

附录 A：词频统计程序

本节的程序统计命令行指定的一组输入文件中每个不同词出现的次数。以下保留原 C++/MapReduce API 结构；仅将注释翻译为中文。

```

#include "mapreduce/mapreduce.h"

// 用户的 map 函数
class WordCounter : public Mapper {
public:
    virtual void Map(const MapInput& input) {
        const string& text = input.value();
        const int n = text.size();
        for (int i = 0; i < n; ) {
            // 跳过开头的空白字符
            while ((i < n) && isspace(text[i])) i++;
            // 找到词尾
            int start = i;
            while ((i < n) && !isspace(text[i])) i++;
            if (start < i) Emit(text.substr(start, i-start), "1");
        }
    }
};
REGISTER_MAPPER(WordCounter);

class Adder : public Reducer {
    virtual void Reduce(ReduceInput* input) {
        // 遍历相同键的所有条目，并将值相加
        int64 value = 0;
        while (!input->done()) {
            value += StringToInt(input->value());
            input->NextValue();
        }
        // 为 input->key() 发射总和
        Emit(IntToString(value));
    }
};
REGISTER_REDUCER(Adder);

int main(int argc, char** argv) {
    ParseCommandLineFlags(argc, argv);
    MapReduceSpecification spec;
    // 将输入文件列表存入 spec
    for (int i = 1; i < argc; i++) {
        MapReduceInput* input = spec.add_input();
        input->set_format("text");
        input->set_filepattern(argv[i]);
        input->set_mapper_class("WordCounter");
    }
    // 指定输出文件: /gfs/test/freq-00000-of-00100, ...
    MapReduceOutput* out = spec.output();
    out->set_filebase("/gfs/test/freq");
    out->set_num_tasks(100);
    out->set_format("text");
    out->set_reducer_class("Adder");
    // 可选: 在 map 任务内部分和, 节约网络带宽
    out->set_combiner_class("Adder");
    // 调优: 最多使用 2,000 台机器, 每个任务 100 MB 内存
    spec.set_machines(2000);
    spec.set_map_megabytes(100);
    spec.set_reduce_megabytes(100);
    MapReduceResult result;
    if (!MapReduce(spec, &result)) abort();
    // result 包含计数器、耗时、所用机器数等信息
    return 0;
}

```

图表中文对照

原论文中的图表承载了架构、吞吐率、尾延迟和采用规模等关键信息。下面保留图形信息，并在每张图下方补充中文标签与含义说明。

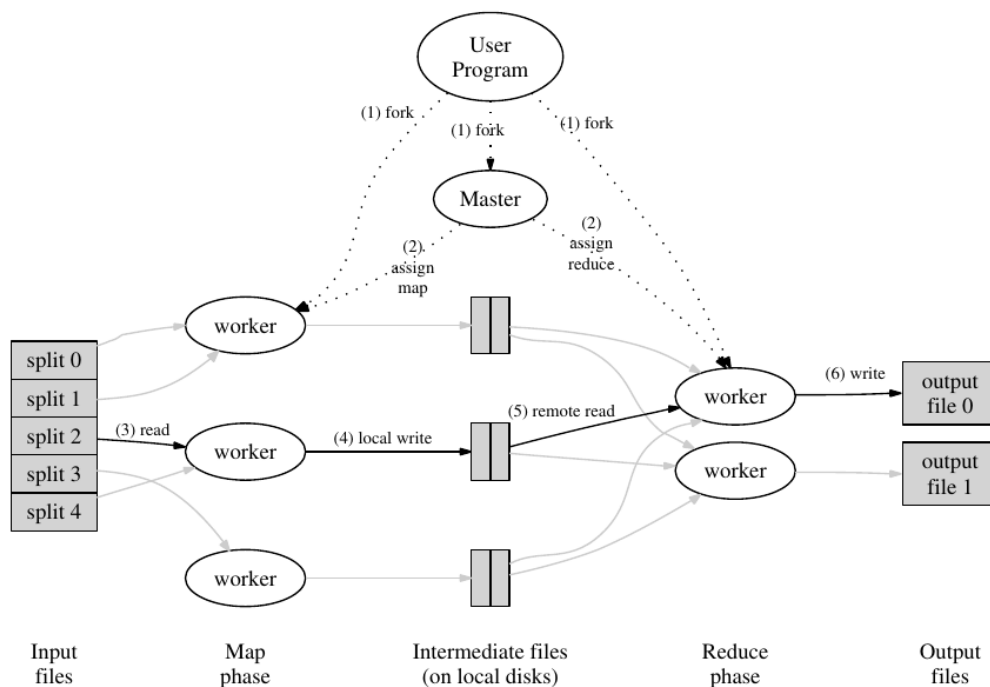


Figure 1: Execution overview

图中文字中文对照：

User Program=用户程序；Master=主节点；worker=工作节点

Input files=输入文件；Map phase=Map 阶段；Intermediate files (on local disks)=中间文件

Reduce phase=Reduce 阶段；Output files=输出文件；fork=创建进程；assign=分配任务

read=读取；local write=本地写入；remote read=远程读取；write=写出；split=输入分片

图 1：执行概览（中文标注）

数据流：输入分片 -> Map -> 本地中间文件 -> Reduce 远程读取/排序 -> 输出文件；Master 负责创建进程和分派任务。

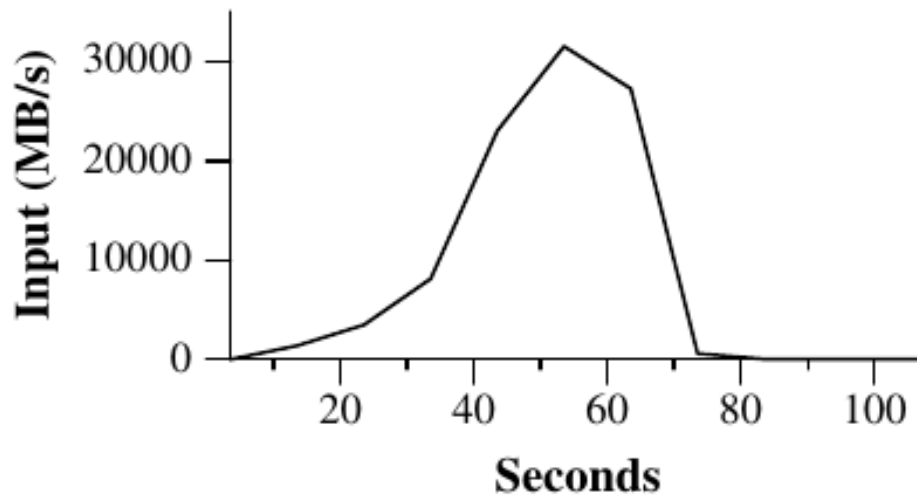


Figure 2: Data transfer rate over time

图中文字中文对照：

Input (MB/s)=输入速率 (MB/s) ; Seconds=

图意：随着 Worker 增加，输入扫描速率上升；

图 2：数据传输速率随时间变化（中文标注）

Grep 计算中，输入读取速率随 Worker 数增加而上升，Map 任务完成后迅速降为零。

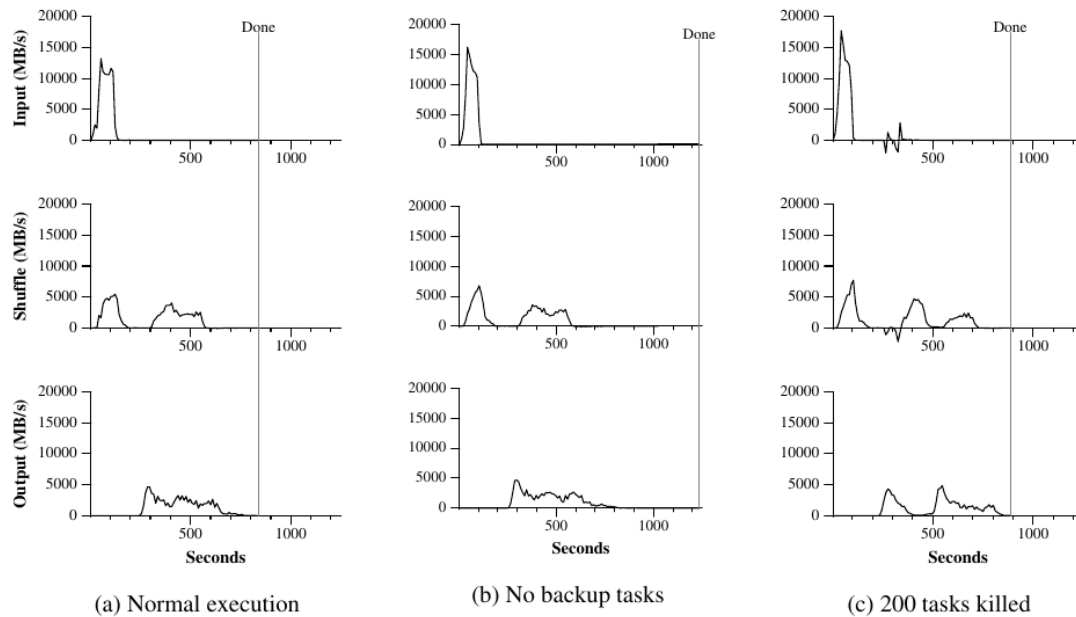


Figure 3: Data transfer rates over time for different executions of the sort program

图中文字中文对照：

Input=输入；Shuffle=重分布（网络搬运）；Output=输出；Done=完成；Seconds=秒

(a) Normal execution=正常执行；(b) No backup tasks=无备份任务；(c) 200 tasks killed=杀
纵轴单位为 MB/s；三列分别展示正常、禁用备份、发生 Worker 故障时的速率曲线。

图 3：排序程序不同执行方式的速率曲线（中文标注）

三种情况：正常执行、禁用备份任务、杀死 200 个 Worker 进程；可观察到 shuffle、输出和长尾的差异。

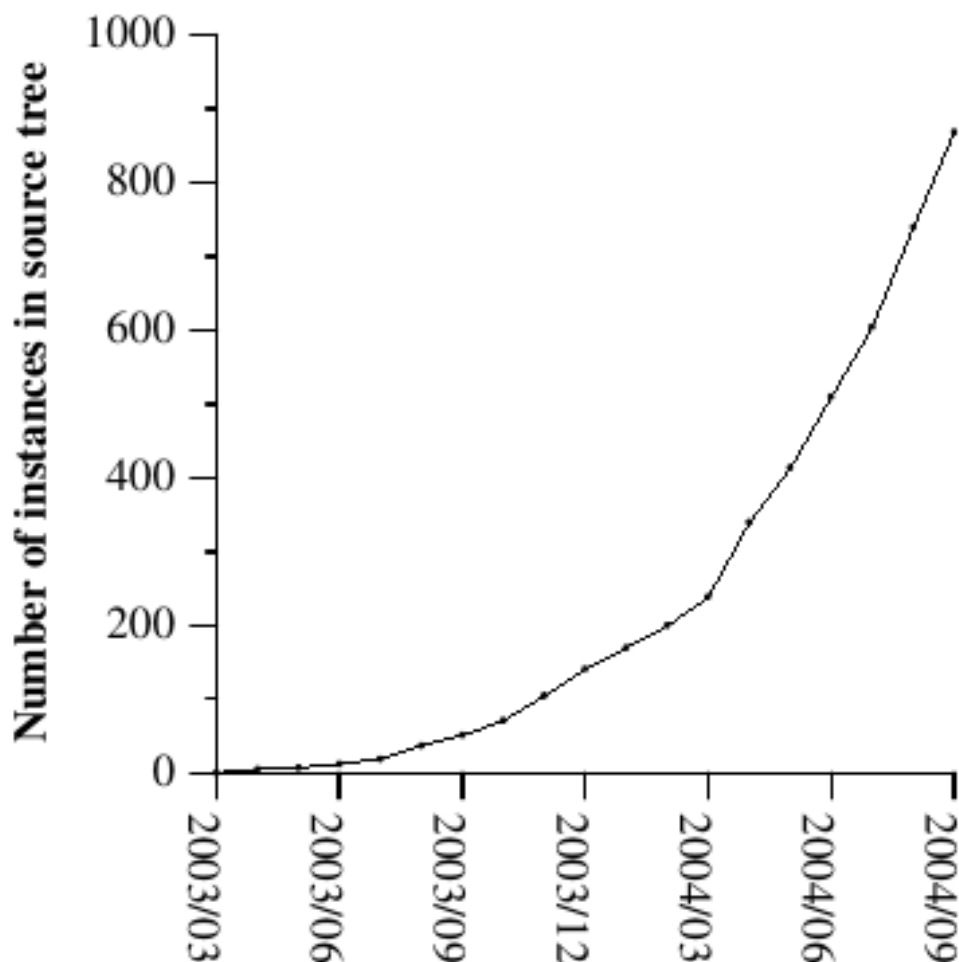


Figure 4: MapReduce instances over time

图中文字中文对照：

Number of instances in source tree=;

图意：MapReduce 程序实例从 2003 年

图 4: MapReduce 实例数量随时间增长 (中文标注)

源码树中的独立 MapReduce 程序实例从 2003 年初的 0 增至 2004 年 9 月末近 900 个。

附：把论文落到今天的学习框架

MapReduce 的原始实现有其时代背景（GFS、廉价 PC、本地磁盘中间文件），但可迁移的设计原则仍是分布式系统基础：以受限接口缩小协调空间；把失败视为常态并采用可重试单元；让数据布局影响计算调度；将慢尾而非平均值作为作业完成时间的主要敌人；以及通过明确输出提交协议给重复执行定义语义。学习后续 Hadoop、Spark、Flink 或云批处理服务时，可以持续拿这些原则作对照：它们主要是在执行模型、内存/磁盘权衡、流处理能力、调度器和状态管理上扩展，而不是推翻这篇论文解决的核心问题。